

CSP Redundancy Mode - Setup Handbook

1. Overview

The CSP redundancy mode enables operating two CSP instances (Cell Service Providers) for the same cell. The goal is uninterrupted continuation of operations in case of an instance failure — even during running recipes.

Concept

- **Primary CSP:** Active instance. Reads and writes OPC-UA items, controls PLC communication, executes recipes and workflows. Only the Primary may write to the database and generate audit entries or messages.
- **Secondary CSP:** Passive instance (standby). Behaves like an RSP — listens to MQTT data, can only trigger workflows via MQTT commands.

Both instances use the same Cell ID. The **ALTERNATIVE node** additionally uses a **static offset of 90,000** at the MQTT topic level — regardless of whether it is currently Primary or Secondary. This allows both CSPs to work simultaneously on the same MQTT broker and the same database without interfering with each other.

Role Distribution

Property	Primary	Secondary
OPC-UA write access to PLC	Yes	Yes
Database write access	Yes	Yes
Generate audit entries / messages	Yes	No
Recipe execution	Active	Passive (takes over on failover)
Publish MQTT data	Yes	Yes
Receive MQTT data	Yes	Yes (listens to Primary topics)
Failover takeover	-	Automatic after 5 seconds

Node Types

There are two node types within a CSP cluster:

- **ORIGINAL** (`cluster.nodeType=1`): The "preferred" instance, publishes under the original Cell ID. When both CSPs start simultaneously, the ORIGINAL node takes precedence in the Primary election.
- **ALTERNATIVE** (`cluster.nodeType=2`): The second instance, publishes under the alternative Cell ID (Cell ID + 90000 offset). Yields to the ORIGINAL node when both claim Primary simultaneously.

Important: The node type only determines priority during election. Both nodes can become either Primary or Secondary.

2. Prerequisites

Hardware

- Two separate machines (or VMs), each running one CSP instance.
- Both machines must have network access.

Network

- Shared **MQTT broker**, reachable from both CSP instances.
- Both CSPs must be able to reach the **PLC via OPC-UA**.
- Both CSPs need access to an **OPC-UA server for cluster communication**. Details in Section 4.
- Both CSPs need access to the database cluster.

PLC-Side Preparation

4 OPC-UA items for cluster communication must be created on the PLC. Details in Section 4.

3. vis.properties Configuration

The `userdata/vis.properties` file must be configured on **both** CSP instances.

3.1 Common Settings (identical on both CSPs)

These properties must be **identical** on both instances:

```
# Cell ID of the cell (identical on both CSPs)
cellbroker.cellId

# MQTT broker URL (same broker for both)
cellbroker.mqttBrokerUrl

# Allowed RSP clients
cellbroker.clientNames

# OPC-UA connection to PLC (same PLC)
OPC.SimaticNet.ipAddress
OPC.SimaticNet.namespaceUri
OPC.SimaticNet.simulated

# Database (same CrateDB instance)
jdbc.ip
jdbc.port
```

3.2 Instance-Specific Settings

These properties **differ** between the two CSP instances:

CSP Instance "Original" (preferred Primary)

```
# Node type: 1 = ORIGINAL (takes precedence in Primary election)
cluster.nodeType=1

# Unique machine name for identification
e.g.: hmi.machineName=CSP 001 Original
```

CSP Instance "Alternative" (standby node)

```
# Node type: 2 = ALTERNATIVE (yields to Original on simultaneous start)
cluster.nodeType=2

# Unique machine name for identification
e.g.: hmi.machineName=CSP 001 Alternative
```

3.3 Cluster Communication (ClusterInfo Items)

Cluster communication runs via 4 OPC-UA items located on an OPC-UA server reachable by both CSPs. The item paths are configured by default via `cluster.*` properties in `vis.properties`. Alternatively, a separate `fspClusterInfo` connection can be used for testing purposes.

Item Paths for Cluster Items (Standard)

The cluster items must be provided on an OPC-UA server. The paths are configured in `vis.properties` and reference items in the AutomationSphere Item Manager — regardless of which connection they are defined on.

```
# Item paths for cluster items
cluster.original.timestamp=OPC.SimaticNet.Cluster.original.timestamp
cluster.original.payload=OPC.SimaticNet.Cluster.original.payload
cluster.alternative.timestamp=OPC.SimaticNet.Cluster.alternative.timestamp
cluster.alternative.payload=OPC.SimaticNet.Cluster.alternative.payload
```

Alternative: fspClusterInfo Connection (Testing Purposes)

If the `cluster.*` properties are **not** set, the default paths on the `fspClusterInfo` connection are used. This variant is suitable for testing purposes and requires a separate OPC-UA connection:

- `Cluster.original.timestamp`
- `Cluster.original.payload`
- `Cluster.alternative.timestamp`
- `Cluster.alternative.payload`

```
# IP address and port of the OPC-UA server for cluster communication
# This is the server where the 4 cluster items reside (see Section 4)
fspClusterInfo.ipAddress=10.80.6.200:4840
```

Note: If `fspClusterInfo.ipAddress` is **not** set, the `fspClusterInfo` connection runs in simulated mode. If neither the `cluster.*` properties nor `fspClusterInfo.ipAddress` are set, the redundancy mode is **not** functional.

3.4 Complete Property Reference

Property	Type	Required	Example	Description
<code>cluster.nodeType</code>	Integer	Yes	1 or 2	1 = C ALTER
<code>cluster.original.timestamp</code>	Long	No	<code>Cluster.original.timestamp</code>	Item p timesta Clust
<code>cluster.original.payload</code>	String(200)	No	<code>Cluster.original.payload</code>	Item p (Defau Clust
<code>cluster.alternative.timestamp</code>	Long	No	<code>Cluster.alternative.timestamp</code>	Item p timesta Clust
<code>cluster.alternative.payload</code>	String(200)	No	<code>Cluster.alternative.payload</code>	Item p payloa Clust
<code>fspClusterInfo.ipAddress</code>	String	No*	10.80.6.200:4840	OPC-U heartb neede are no
<code>cellbroker.cellId</code>	Integer	Yes	815	Cell ID
<code>cellbroker.mqttBrokerUrl</code>	String	Yes	<code>tcp://192.168.1.100:1883</code>	MQTT

Property	Type	Required	Example	Description
<code>hmi.machineName</code>	String	Recommended	CSP 815 Original	Display name of the CSP

4. OPC-UA Items on the PLC (Cluster Communication)

4.1 Overview

The two CSP instances communicate their status (Primary/Secondary) and heartbeat signals via **4 OPC-UA items**. These items must reside on an OPC-UA server reachable by both CSPs.

The items are arranged in pairs — one pair for the ORIGINAL node and one for the ALTERNATIVE node:

Item Identifier	OPC-UA Data Type	Access	Description
<code>Cluster.original.timestamp</code>	UInt64	Read/Write	Heartbeat timestamp of the ORIGINAL node (milliseconds since epoch)
<code>Cluster.original.payload</code>	String(200)	Read/Write	JSON payload of the ORIGINAL node (mode and command, see 4.2)
<code>Cluster.alternative.timestamp</code>	UInt64	Read/Write	Heartbeat timestamp of the ALTERNATIVE node (milliseconds since epoch)
<code>Cluster.alternative.payload</code>	String(200)	Read/Write	JSON payload of the ALTERNATIVE node (mode and command, see 4.2)

4.2 Payload Format (JSON)

The payload item contains a JSON string with the following structure:

```
{
  "mode": "PRIMARY",
  "command": "REQUEST_PRIMARY_STATE"
}
```

Possible values for `mode` :

Value	Meaning
PRIMARY	Instance is active Primary
SECONDARY	Instance is on standby
REQUEST_BECOMING_PRIMARY	Instance wants to become Primary
TRANSITION_TO_PRIMARY	Transition to Primary in progress
REQUEST_BECOMING_SECONDARY	Instance switching to Secondary
TRANSITION_TO_SECONDARY	Transition to Secondary in progress

Possible values for `command` :

Value	Meaning
REQUEST_PRIMARY_STATE	Request: May I become Primary?
DENY_PRIMARY_STATE	Denial: No, I am Primary
GRANT_PRIMARY_STATE	Approval: Yes, take over

4.3 Heartbeat Mechanism

- Each CSP instance writes its current timestamp (milliseconds since epoch) to its own `timestamp` item **every second**.
 - The other instance reads this timestamp and detects via a **5-second threshold** whether the remote side is still alive.
 - If the heartbeat of an instance is missing for more than 5 seconds, a failover is initiated.
-

5. clusterSyncItems - Synchronized Items

5.1 Purpose

The `clusterSyncItems` group defines which items are synchronized between Primary and Secondary. When the Secondary becomes Primary (failover), it takes over the current values of these items from the previous Primary.

5.2 Standard Items

The following items are synchronized by default in the `asphere.xml` :

Batch and Operations Items:

Item Name	Description
<code>hmi.batch.batchId</code>	Current batch ID
<code>hmi.batch.phaseId</code>	Current phase ID
<code>hmi.operation.operationId</code>	Current operation ID
<code>hmi.operation.recordingId</code>	Current recording ID

Timer Items (synchronized automatically, details in Section 7.4):

Item Group	Items	Description
Start Booleans	<code>csp.timer.start1/2/3</code>	Timer running status
Stop Booleans	<code>csp.timer.stop1/2/3</code>	Timer stop status
Reset Booleans	<code>csp.timer.reset1/2/3</code>	Timer reset status
Elapsed Millis	<code>csp.timer.timeMilliseconds1/2/3</code>	Elapsed milliseconds
Names	<code>csp.timer.name1/2/3</code>	Timer labels

5.2.1 Programmatically Registered Items

In addition to the items declared in the `asphere.xml`, further items are programmatically added to the `clusterSyncItems` group at runtime. These do **not** need to be configured manually:

Profile Items (`ProfileHandler`): For each configured profile (e.g., sensors, actuators, valves), three items are automatically registered:

Item	Description
<code>*.Profil.SP_RExecTime</code> / <code>*.Profil.SP-EU_RExecTime</code>	Profile execution time
<code>*.Profil.SP_RState</code> / <code>*.Profil.SP-EU_RState</code>	Profile status
<code>*.Profil.SP_RProfileName</code> / <code>*.Profil.SP-EU_RProfileName</code>	Active profile name

Note: The number of profile items depends on the plant configuration. For a typical cell with 10 profiles, 30 additional items are synchronized.

5.3 Synchronization Mechanism

- In **Primary mode**: Local items (NullItems) are active and written locally. No synchronization needed.
- In **Secondary mode**: The local NullItems are linked with the corresponding remote items (BridgeItems) of the Primary (master binding). Changes from the Primary are automatically adopted via MQTT.
- During **failover** (Before-Disconnect pattern):
 - The `ClusterItemSynchronizationHandler` detects the switch to PRIMARY.
 - **Before** removing the master bindings, all registered `beforeMasterDisconnect` consumers are called. These can read the current values from the NullItems while the master bindings are still active.
 - Only then are the master bindings removed (`setMasterItem(null)`).
 - The new Primary continues working with the previously saved values.

Technical Background: The Before-Disconnect pattern is necessary because removing the master bindings resets the NullItem values to their defaults.

Consumers like the `AlternativeTimerController` must read their values before this happens.

5.4 Adding Further Items

If additional items need to be synchronized, they should be added to the `clusterSyncItems` subGroup.

6. Commissioning

6.1 Pre-Start Checklist

- `[]` OPC-UA items created on the PLC/OPC-UA server (4 items, see Section 4)
- `[] vis.properties` configured on CSP "Original" (`cluster.nodeType=1`)
- `[] vis.properties` configured on CSP "Alternative" (`cluster.nodeType=2`)
- `[]` Cluster connection configured (one of the two options):
- **Standard:** `cluster.original.*` and `cluster.alternative.*` properties set on both CSPs + items present on the respective OPC-UA server
- **Testing:** `fspClusterInfo.ipAddress` set on both CSPs (fallback, if `cluster.*` properties are not set)
- `[] cellbroker.cellId` identical on both CSPs
- `[] cellbroker.mqttBrokerUrl` identical on both CSPs
- `[] asphere.xml` identical on both CSPs
- `[]` MQTT broker reachable from both CSPs
- `[]` OPC-UA server (PLC) reachable from both CSPs
- `[]` OPC-UA server (cluster items) reachable from both CSPs (variant A only)
- `[]` CrateDB reachable from both CSPs
- `[] app.db.backupscheduler=true` set on **both** CSPs (see Section 7.6)

6.2 Startup Order

1. **Start MQTT broker** (if not already running)
2. **Start CrateDB** (if not already running)
3. **Start CSP "Original"** (`cluster.nodeType=1`)
4. On startup, this instance attempts to acquire Primary status.
5. After 5 seconds without objection, it becomes Primary.

6. **Start CSP "Alternative"** (cluster.nodeType=2)
7. This instance detects that the Original node is already Primary.
8. It automatically becomes Secondary.

Note: The startup order is not mandatory. Even with simultaneous startup, the ORIGINAL node is preferred as Primary. The election mechanism is deterministic.

6.3 Post-Start Verification

Check on Primary CSP:

- `hmi.machineName` is displayed in the UI (e.g., "CSP 815 Original")
- MQTT topic `STATE/CSP815/APP/appmode` contains the value `PRIMARY`
- OPC-UA heartbeat: `Cluster.CSP815.original.timestamp` is updated every second
- OPC-UA payload: `Cluster.CSP815.original.payload` shows `"mode": "PRIMARY"`

Check on Secondary CSP:

- MQTT topic `STATE/CSP90815/APP/appmode` contains the value `SECONDARY`
- OPC-UA heartbeat: `Cluster.CSP815.alternative.timestamp` is updated every second
- OPC-UA payload: `Cluster.CSP815.alternative.payload` shows `"mode": "SECONDARY"`

Note on offset: The ALTERNATIVE node publishes its MQTT topics **always** with Cell ID + 90,000, regardless of whether it is Primary or Secondary. For Cell ID `815`, `CSP90815` is used as the client name.

7. Failover Behavior

7.1 Automatic Failover

The failover is triggered automatically when the Secondary detects that the Primary's heartbeat has been **missing for more than 5 seconds**.

Sequence:

1. Primary CSP fails (process crash, network failure, etc.)
2. The Primary's heartbeat timestamp is no longer updated
3. After 5 seconds, the Secondary detects the failure
4. The Secondary switches to `TRANSITION_TO_PRIMARY` mode
5. The Secondary becomes the new Primary
6. `ClusterItemSynchronizationHandler` detects the switch to PRIMARY: a. **Before-Disconnect phase:** Registered consumers (e.g., `AlternativeTimerController`) read the current values from the NullItems while the master bindings are still active. b. **Disconnect phase:** The master bindings are removed (`setMasterItem(null)`).
7. Running recipes and operations are continued.

7.1.1 Primary Self-Monitoring

The Primary monitors not only the Secondary's heartbeat but also its **own** timestamp. At periodic intervals, it checks whether its last written heartbeat is still current. If it determines that its own timestamp is stale (e.g., due to a temporary network failure to the OPC-UA server), it assumes that the Secondary has taken over the Primary role in the meantime. In this case, the Primary initiates an **automatic shutdown** to prevent a state with two simultaneous Primaries. After a restart (manual or via the Windows service), the former Primary returns as Secondary.

7.2 Timeline

t=0s	Primary failure
t=1s	Heartbeat missing (1st check)
t=2s	Heartbeat missing (2nd check)
t=3s	Heartbeat missing (3rd check)
t=4s	Heartbeat missing (4th check)
t=5s	Threshold exceeded -> Failover starts
t=~6s	Secondary is new Primary

7.3 Recipe Failover (Hot Start)

When a recipe is running and the Primary fails, the new Primary performs a **hot start**:

- The Secondary has been continuously reading the recipe state via MQTT during normal operation.

- When switching to Primary, the instance detects that a recipe was active on the other side but none is running locally.
- The recipe is automatically resumed from the last known state — no manual restart is required.
- The PLC-side logic continues running independently; the new Primary takes over communication.
- Batch operations (audit entries, measurement recording) are written directly to the database after the switch instead of being delegated via MQTT.

7.4 Timer Behavior on Failover

The three SCADA timers are continuously synchronized between Primary and Secondary via the `clusterSyncItems` group (running status, elapsed time, timer name). On failover, the new Primary automatically reconstructs the internal timer state from the last synchronized values. Running timers continue counting seamlessly; stopped timers retain their elapsed time. The timer display in the UI updates without manual intervention.

7.5 Profile Behavior on Failover

Profile states (execution time, status, active profile name) are synchronized via the `clusterSyncItems` group and are immediately available to the new Primary after a failover. Reloading or reactivating profiles is not necessary.

7.6 Snapshot Behavior on Failover

In **Secondary mode**, manual snapshot requests are delegated to the Primary via MQTT command — the Secondary does not create database snapshots directly. After a failover, the new Primary takes over this role. Scheduled snapshots then run only on the new Primary.

Important: The property `app.db.backupscheduler` must be set to `true` on **both** CSP instances. The Quartz scheduler is only created on instances where this property is enabled. If it is set to `true` on only one CSP, no snapshots will be created after a failover.

7.7 Return of the Original Node

Scenario A: Restart After Crash

When the failed Original node restarts:

1. It requests Primary status (`REQUEST_PRIMARY_STATE`).
2. The Alternative node is already a healthy Primary and **denies** the request (`DENY_PRIMARY_STATE`).
3. The Original node accepts the denial and becomes **Secondary**.

The ORIGINAL node has precedence **only during simultaneous startup** of both instances. When a healthy Primary already exists, the returning node — regardless of node type — always becomes Secondary.

Scenario B: Temporary Connection Loss

If the Original node does not crash but merely temporarily loses connection to the OPC-UA server:

1. The self-monitoring (see Section 7.1.1) detects the stale own timestamp.
 2. The Primary initiates an **automatic shutdown**.
 3. After a restart (manual or via the Windows service), the node returns as Secondary.
-

7.8 RSP Behavior on Failover

Connected RSP instances (SCADA clients) notice the role change through a status change on the MQTT topic `STATE/<cellId>/APP/appmode` . However, for the user at the RSP **nothing changes** — the user interface remains functional as the new Primary seamlessly takes over MQTT communication. The RSP continues sending commands to the same Cell ID; these are processed by the currently active Primary.

7.9 Single-Instance Operation Without Redundancy

Single-instance operation without redundancy can be achieved by simply running only one CSP instance. In this case:

- The CSP starts as an ORIGINAL node and requests Primary status.
- After 5 seconds without a response from a counterpart, it automatically becomes Primary.

- The ClusterManager runs in the background but has no effect on operations.
- No offset is applied (only the ALTERNATIVE node uses the 90,000 offset).
- All functions (recipes, timers, snapshots, database) work as in non-redundancy mode.

Minimal configuration for single-instance operation:

No cluster-specific properties need to be set. The following properties can be omitted:

```
# NOT required for single-instance operation:  
# cluster.nodeType           (Default: ORIGINAL)  
# cluster.original.*        (not needed)  
# cluster.alternative.*     (not needed)  
# fspClusterInfo ipAddress  (not needed)
```